باسمه تعالی

آزمایشگاه طراحی سیستم‌های دیجیتال

دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

تابستان ۱۴۰۳

استاد:

دکتر محسن انصاری

مهندس محمدرضا حسینی

اعضای گروه:

سعید فراتی کاشانی – ۴۰۱۱۰۶۲۹۹

امیرحسین صوری – ۴۰۱۱۰۶۱۸۲

محمدامین عباس‌فر – ۴۰۱۱۰۶۲۱۱

فهرست عناوین

موضوع آزمایش: ۳
شرح کلی آزمایش: ۳
پیاده‌سازی بخش اول (مقایسه‌کننده ۴ بیتی): ۳
طراحی Single Bit Comparator ۳
طراحی ۴–Bit Adder ٤
طراحی ماژول Test Bench ٥
پیاده‌سازی بخش دوم (مقایسه‌کننده سریال): ۷
طراحی Serial Comparator ۷
طراحی ماژول Test Bench ۸

موضوع آزمایش:

توصیف جریان داده

شرح کلی آزمایش:

در این آزمایش می‌خواهیم دو عدد باینری را مقایسه کنیم. در بخش اول آن ۴ مقایسه‌کننده تک بیتی می‌سازیم و با متصل کردن آن‌ها به صورت یک مدار ترکیبی، مقایسه‌کننده ۴ بیتی‌مان را شکل می‌دهیم. در بخش دوم، با استفاده از توصیف جریان داده یک مقایسه‌کننده سریالی می‌سازیم.

پیاده‌سازی بخش اول (مقایسه‌کننده ۴ بیتی):

برای این آزمایش ابتدا یک مقایسه‌کننده تک بیتی می‌سازیم؛ سپس با اتصال چهار نمونه از آن به یک‌دیگر، مقایسه‌کننده ۴ بیتی‌مان را شکل می‌دهیم.

طراحی Single Bit Comparator

در گام اول یک مقایسه‌کننده به صورت زیر پیاده‌سازی کردیم. اگر بیت جدید در عدد A، بزرگتر و یا کوچک‌تر از بیت جدید در عدد B باشد، یکی از سیگنال‌های gt یا lt را ۱ می‌کنیم. در صورتی که بیت جدید هر دو برابر با ۱ بود، مقدار lt و gt برابر با همان حالت قبلی‌شان که از ورودی گرفته می‌شود، خواهد بود. در صورتی که نه lt رخ دهد و نه gt، یعنی اعداد برابرند و در نتیجه سیگنال eq را فعال خواهیم کرد.

```
module single_bit_comparator (
    input wire A,
    input wire B,
    input wire A_gt_B_in,
    input wire A_lt_B_in,
    output wire A_gt_B,
    output wire A_lt_B,
    output wire A_eq_B
);

assign A_gt_B = (A & ~B) | (A_gt_B_in & ~(A ^ B));
assign A_lt_B = (~A & B) | (A_lt_B_in & ~(A ^ B));
assign A_eq_B = ~(A_gt_B | A_lt_B);

endmodule
```

طراحی ۴–Bit Adder

در گام بعدی، ۴ مقایسه‌کننده تک‌بیتی را به یک‌دیگر متصل کردیم. همچنین تعدادی wire داخل ماژول تعریف کردیم تا بتوانیم مقادیر سیگنال‌های مربوط به مقایسه بیت قبلی را به مقایسه‌کننده‌ی متعلق به بیت جدید بدهیم.

```
module four_bit_comparator (
    input wire [0:3] A,
    input wire [0:3] B,
    output wire A_gt_B,
    output wire A_lt_B,
    output wire A_eq_B
);

wire [2:0] A_gt_B_internal, A_lt_B_internal, A_eq_B_internal;

single_bit_comparator comp0 (
    .A(A[3]),
    .B(B[3]),
    .A_gt_B_in(1'b0),
    .A_lt_B_in(1'b0),
    .A_gt_B(A_gt_B_internal[2]),
    .A_lt_B(A_lt_B_internal[2]),
    .A_eq_B(A_eq_B_internal[2])
);

single_bit_comparator comp1 (
    .A(A[2]),
    .B(B[2]),
    .A_gt_B_in(A_gt_B_internal[2]),
    .A_lt_B_in(A_lt_B_internal[2]),
    .A_gt_B(A_gt_B_internal[1]),
    .A_lt_B(A_lt_B_internal[1]),
    .A_eq_B(A_eq_B_internal[1])
);

single_bit_comparator comp2 (
    .A(A[1]),
    .B(B[1]),
    .A_gt_B_in(A_gt_B_internal[1]),
    .A_lt_B_in(A_lt_B_internal[1]),
    .A_gt_B(A_gt_B_internal[0]),
    .A_lt_B(A_lt_B_internal[0]),
    .A_eq_B(A_eq_B_internal[0])
);

single_bit_comparator comp3 (
    .A(A[0]),
    .B(B[0]),
    .A_gt_B_in(A_gt_B_internal[0]),
    .A_lt_B_in(A_lt_B_internal[0]),
    .A_gt_B(A_gt_B),
    .A_lt_B(A_lt_B),
    .A_eq_B(A_eq_B)
);

endmodule
```

طراحی ماژول Test Bench

حال، برای بررسی صحت عملکرد مقایسه‌کننده‌ای که طراحی کردیم، ماژول آزمون زیر را آماده کردیم:

```
module four_bit_comparator_tb;

reg [3:0] A, B;
wire A_gt_B, A_lt_B, A_eq_B;

four_bit_comparator uut (
    .A(A),
    .B(B),
    .A_gt_B(A_gt_B),
    .A_lt_B(A_lt_B),
    .A_eq_B(A_eq_B)
);

initial begin
    $monitor("A=%b B=%b --> A_gt_B=%b A_lt_B=%b A_eq_B=%b",
     A, B, A_gt_B, A_lt_B, A_eq_B);

    A = 4'b0000; B = 4'b0000;
    #10;

    A = 4'b1001; B = 4'b0110;
    #10;

    A = 4'b0010; B = 4'b1010;
    #10;

    A = 4'b1111; B = 4'b1111;
    #10;

    A = 4'b0111; B = 4'b1000;
    #10;

    $finish;
end

endmodule
```

همان‌طور که مشاهده می‌کنید، در حالت اول برابرند. در حالت دوم A بزرگ‌تر از B است. در حالت سوم A کوچک‌تر از B است. در حالت چهارم A برابر با B است و در نهایت در حالت پنجم، B بزرگ‌تر از A است. حال بررسی می‌کنیم که آیا خروجی مطابق انتظار ما شده است یا نه.

```
PS D:\> iverilog -o out dsd-q3-p1.v
PS D:\> vvp out
A=0000 B=0000 --> A_gt_B=0 | A_lt_B=0 | A_eq_B=1
A=1001 B=0110 --> A_gt_B=1 | A_lt_B=0 | A_eq_B=0
A=0010 B=1010 --> A_gt_B=0 | A_lt_B=1 | A_eq_B=0
A=1111 B=1111 --> A_gt_B=0 | A_lt_B=0 | A_eq_B=1
A=0111 B=1000 --> A_gt_B=0 | A_lt_B=1 | A_eq_B=0
dsd-q3-p1.v:101: $finish called at 50 (1s)
```

همان‌طور که انتظار داشتیم، خروجی‌ها به طور صحیح محاسبه شدند پس می‌توانیم نسبت به صحت عملکرد ماژول طراحی شده، اطمینان پیدا کنیم.

پیاده‌سازی بخش دوم (مقایسه‌کننده سریال):

در این بخش از آزمایش، بیت‌ها را به صورت سریالی ورودی می‌گیریم و آن‌ها را مقایسه می‌کنیم.

طراحی Serial Comparator

برای پیاده‌سازی این بخش، مشابه حالت قبلی عمل کردیم با این تفاوت که به‌جای ورودی گرفتن وضعیت سیگنال‌ها در حالت قبلی، آن‌ها را در reg نگه می‌داریم تا در صورت نیاز، مورد استفاده قرار بگیرند. همچنین برای مقداردهی اولیه، از سیگنال reset استفاده کردیم تا سیگنال‌ها مقداردهی اولیه شوند و برابر با X نمانند.

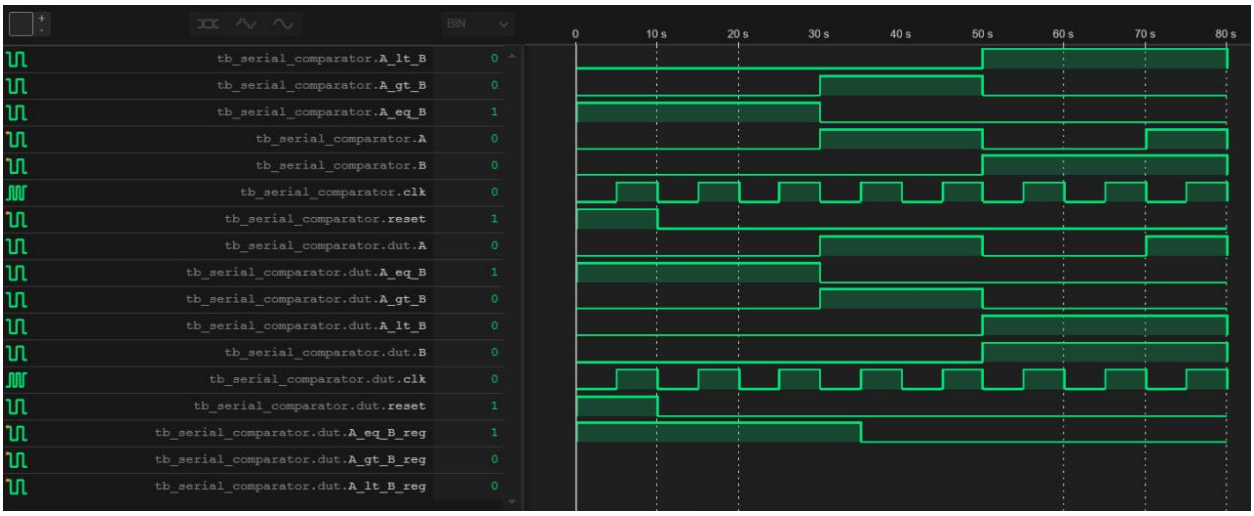
```
module serial_comparator (
    input wire clk,
    input wire reset,
    input wire A,
    input wire B,
    output wire A_gt_B,
    output wire A_lt_B,
    output wire A_eq_B
);

reg A_gt_B_reg, A_lt_B_reg, A_eq_B_reg;

assign A_gt_B = (A & ~B) | (A_gt_B_reg & ~(A ^ B));
assign A_lt_B = (~A & B) | (A_lt_B_reg & ~(A ^ B));
assign A_eq_B = ~(A_gt_B | A_lt_B);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        A_gt_B_reg <= 0;
        A_lt_B_reg <= 0;
        A_eq_B_reg <= 1;
    end else begin
        A_gt_B_reg <= A_gt_B;
        A_lt_B_reg <= A_lt_B;
        A_eq_B_reg <= A_eq_B;
    end

end

endmodule
```

طراحی ماژول Test Bench

برای بررسی صحت عملکرد مقایسه‌کننده‌ی سریالی‌مان، ماژول محک زیر را طراحی کردیم:

```
module serial_comparator_tb();

    reg clk;
    reg reset;
    reg A, B;
    wire A_gt_B, A_lt_B, A_eq_B;

    serial_comparator dut (
        .clk(clk),
        .reset(reset),
        .A(A),
        .B(B),
        .A_gt_B(A_gt_B),
        .A_lt_B(A_lt_B),
        .A_eq_B(A_eq_B)
    );

    always #5 clk = ~clk;

    initial
    $monitor("Time=%2t, A=%b, B=%b --> A_gt_B=%b | A_lt_B=%b | A_eq_B=%b | reset=%b",
                 $time, A, B, A_gt_B, A_lt_B, A_eq_B, reset);


    initial begin

        $dumpfile("sim_results.vcd");
        $dumpvars(0, serial_comparator_tb);

        clk = 0;
        reset = 1;

        A = 1'b0; B = 1'b0;
        #10 reset = 0;

        #20 A = 1'b1; B = 1'b0;

        #20 A = 1'b0; B = 1'b1;

        #20 A = 1'b1; B = 1'b1;

        #10 $finish;

    end

endmodule
```

ابتدا هر دو بیت ۰ هستند پس اعداد با یک‌دیگر برابرند. در گام بعدی بیت مربوط به A یک می‌شود در حالی که بیت مربوط به B صفر است پس A بزرگ‌تر از B می‌شود. در گام بعدی بیت پر ارزش جدید متعلق به B است پس A کوچک‌تر از B می‌شود. در گام نهایی نیز هر دو بیت جدید ۱ هستند پس در همان وضعیت قبلی یعنی A کوچک‌تر از B باقی می‌ماند. حال بررسی می‌کنیم که آیا خروجی مطابق انتظارمان است یا نه.

```
PS D:\> iverilog -o out dsd-q3-p2.v
PS D:\> vvp out
VCD info: dumpfile sim_results.vcd opened for output.
Time= 0, A=0, B=0 --> A_gt_B=0 | A_lt_B=0 | A_eq_B=1 | reset=1
Time=10, A=0, B=0 --> A_gt_B=0 | A_lt_B=0 | A_eq_B=1 | reset=0
Time=30, A=1, B=0 --> A_gt_B=1 | A_lt_B=0 | A_eq_B=0 | reset=0
Time=50, A=0, B=1 --> A_gt_B=0 | A_lt_B=1 | A_eq_B=0 | reset=0
Time=70, A=1, B=1 --> A_gt_B=0 | A_lt_B=1 | A_eq_B=0 | reset=0
dsd-q3-p2.v:74: $finish called at 80 (1s)
```

همان‌طور که انتظار داشتیم، خروجی‌ها به درستی محاسبه شدند. سیگنال‌های حین اجرای ماژول محک را نیز در تصویر زیر می‌توانیم مشاهده کنیم که همگی به درستی مقداردهی شده‌اند.